6th Jun, 2025

1 Generating a phantom and a sinogram

- Generate a phantom where each pixel consists of part bone and part water described as (bone, water)
  - Background values should be low for both to simulate air (0, 0.05)
  - main body should be elliptical and mainly consist of water (0.20, 0.80)
  - elliptical inserts should simulate different organs and/or body featurs (0.80, 0.80)
- Generate a sinogram from the projections
  - The function will return a 3D object containing the sinograms for the height of the detector. We want to select a slice to display
  - print a 2D cross section of the phantom with a corresponding sinogram

```
#settings
objectSize = 32     # size of the object
nPixelsY = 64       # number of pixels in Y direction detector
nPixelsZ = 44       # number of pixels in Z direction detector
projections = 64   # number of projections
```

1.0.1 generate phantom

```
import random, numpy as np

def generate_ct_matrices(n, h, num_features=5):

    # Initialize 3D bone and water matrices with zeros
    bone_matrix = np.zeros((n, n, n), dtype=float)
    water_matrix = np.zeros((n, n, n), dtype=float)

    # fill background of matrices with 0.05 bone and water
    bone_matrix.fill(0)
    water_matrix.fill(0.05)

    # set values for main ellipse (simulated torso)
    intensity_bone = 0.2
    intensity_water = 0.8

    # Main body ellipse (simulated torso)
    for i in range(h, n - h):
        for j in range(2 * h, n - 2 * h):
            if ((i - n//2) / ((n - 2 * h) / 2)) ** 2 + ((j - n//2) / ((n - 6 * h) / 2)) ** 2 <= 1:

                bone_matrix[2*h: -2*h, i, j] = intensity_bone
                water_matrix[2*h: -2*h, i, j] = intensity_water

    # Adding smaller features (e.g., organs or bone structures)
    for _ in range(num_features):

        # Random place in the body - h is the offset from the edge, n is the size of the matrix
        a = random.randint(h, (n - h) // 4)  # Semi -major axis
        b = random.randint(h, (n - h) // 4)  # Semi -minor axis
        center_x = random.randint(h + a, n - h - a)
        center_y = random.randint(3 * h + b, n - 3 * h - b)

        # Randomly choose the intensity for bone and water, together they add to 1.
```

```
        bone_intensity = 0.8
        water_intensity = 0.8

        for i in range(center_x - a, center_x + a):
            for j in range(center_y - b, center_y + b):
                if 0 <= i < n and 0 <= j < n:
                    if ((i - center_x) / a) ** 2 + ((j - center_y) / b) ** 2 <= 1:
                        bone_matrix[2*h: -2*h, i, j] = bone_intensity
                        water_matrix[2*h: -2*h, i, j] = water_intensity
    water_matrix *= 1  # Water density is 1
    bone_matrix *= 1.92  # Apply bone density

    return bone_matrix, water_matrix

bone, water = generate_ct_matrices(objectSize, 2, num_features=3)
x = np.column_stack((bone.ravel(), water.ravel()))
```

1.0.2 Running projection algorithm

```
from libs.simulatepreps import projectMatrix

#generate proection matrices
y, S, M, A = projectMatrix(x, objectSize, nPixelsY, nPixelsZ, 1, projections)
```

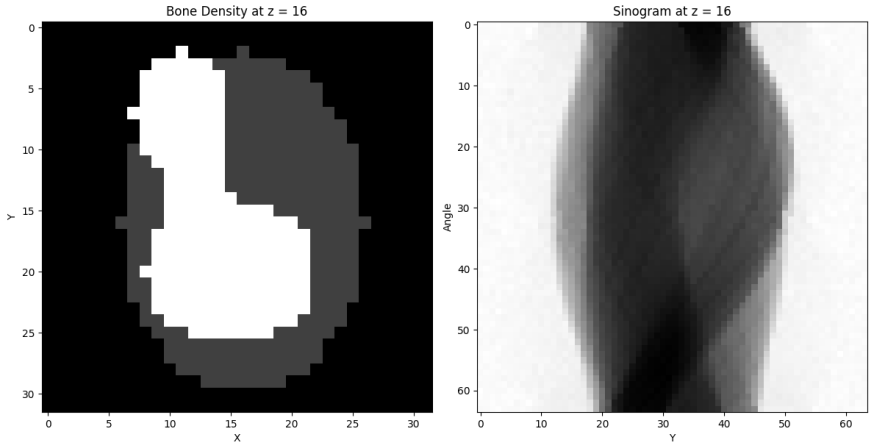
1.0.3 Generating sinogram

```
def getSinogram(y, energy_bin_index=0, z_index=32, nProj=100, detector_shape=(64, 44)):
    nX, nZ = detector_shape
    y_bin = y[energy_bin_index]
    y_proj_zx = y_bin.reshape((nProj, nZ, nX))  # shape: (100, 64, 44)
    sinogram = y_proj_zx[:, z_index, :]  # shape: (100, 44)
    return sinogram

sinogram = getSinogram(y, energy_bin_index=0, z_index=objectSize // 2, nProj=projections, detector_shap
```

1.0.4 Make figure

```
import matplotlib.pyplot as plt


# plot bone density and phantom at objectSize //2
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.imshow(bone[objectSize // 2, :, :], cmap='gray')
plt.title('Bone Density at z = {}'.format(objectSize // 2))
plt.xlabel('X')
plt.ylabel('Y')

plt.subplot(1, 2, 2)
plt.imshow(sinogram, cmap='gray')
plt.title('Sinogram at z = {}'.format(objectSize // 2))
plt.ylabel('Angle')
plt.xlabel('Y')

plt.tight_layout()
```

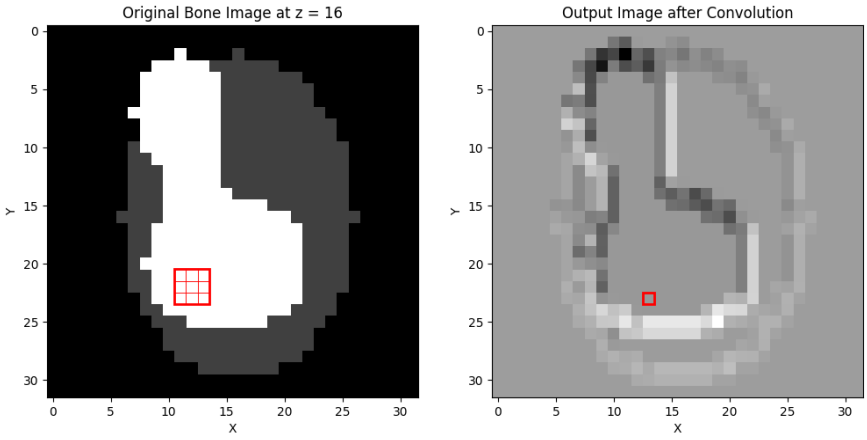
```
plt.show()
```



1.1 Visualising a 2D convolution

```
# we want to put the phantom through a pytorch 2d convolution layer
import torch

bone_array = bone[objectSize // 2, :, :]
bone_tensor = torch.tensor(bone[objectSize // 2, :, :], dtype=torch.float32)

# create a 2D convolution layer with a kernel size of 3x3
conv_layer = torch.nn.Conv2d(in_channels=1, out_channels=1, kernel_size=3, padding=1)

# apply the convolution layer to the bone tensor
output_tensor = conv_layer(bone_tensor.unsqueeze(0).unsqueeze(0))  # Add batch and channel dimensions
output_image = output_tensor.squeeze(0).squeeze(0).detach().numpy()  # Remove batch and channel dimensi

# plot 2 plots, one of the original bone image with a square showing one of the convolution kernels, an
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.imshow(bone_tensor.numpy(), cmap='gray')
# draw a square representing the convolution kernel
kernel_size = conv_layer.kernel_size[0]
plt.gca().add_patch(plt.Rectangle((10.5, 20.5), kernel_size, kernel_size, edgecolor='red', facecolor='n
# fill square with rectengles for each pixel
for i in range(kernel_size):
    for j in range(kernel_size):
        plt.gca().add_patch(plt.Rectangle((10.5 + i, 20.5 + j), 1, 1, edgecolor='red', facecolor='none'
plt.title('Original Bone Image at z = {}'.format(objectSize // 2))
plt.xlabel('X')
plt.ylabel('Y')
plt.subplot(1, 2, 2)
plt.imshow(output_image, cmap='gray')
# draw little square representing the pixel on which the convolution was applied
plt.gca().add_patch(plt.Rectangle((10.5+2, 20.5+2), 1, 1, edgecolor='red', facecolor='none', lw=2))
plt.title('Output Image after Convolution')
plt.xlabel('X')
plt.ylabel('Y')
```

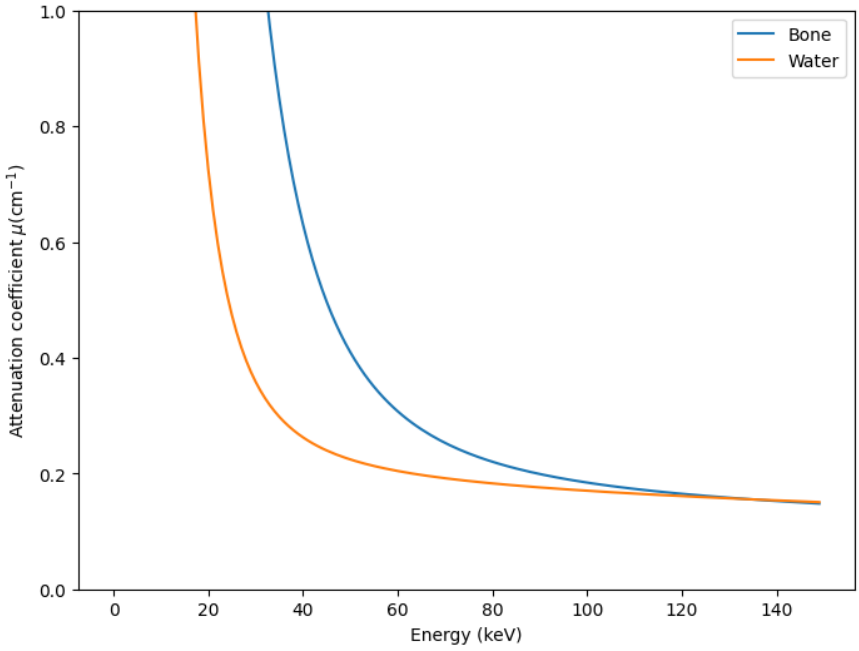
```
Text(0, 0.5, 'Y')
```


Original Bone Image at z = 16
Output Image after Convolution
Y
X
0
5
10
15
20
25
30

2 Load attenuation coefficients

The attenuation coefficients change depending on the energy of the photons used. Load values and plot these.

```
import scipy.io
import matplotlib.pyplot as plt

# Load the .mat file
mat_data = scipy.io.loadmat('libs/materialAttenuationBoneWater.mat')

first = mat_data['materialAttenuations'][:,0]
second = mat_data['materialAttenuations'][:,1]
plt.figure(figsize=(8, 6))
plt.plot(first, label='Bone')
plt.plot(second, label='Water')
plt.xlabel('Energy (keV)')
#nicely format the unit
plt.ylabel(r"Attenuation coefficient $\mu$(cm$^{ -1}$)")
plt.legend()
plt.ylim(0,1)
plt.show()
```



3 Generate callibration data

The goal of the callibration data is to pre-train the network by mapping parts of the sinogram to corresponding pixels.

The data will consist of a few squares of bone and water on an empty background. The network will be trained on input/output with a single step so no reconstruction algorithm is needed for this data.

The network will be trained with only a single step, as the data should be easy to interpret with very little information per image.

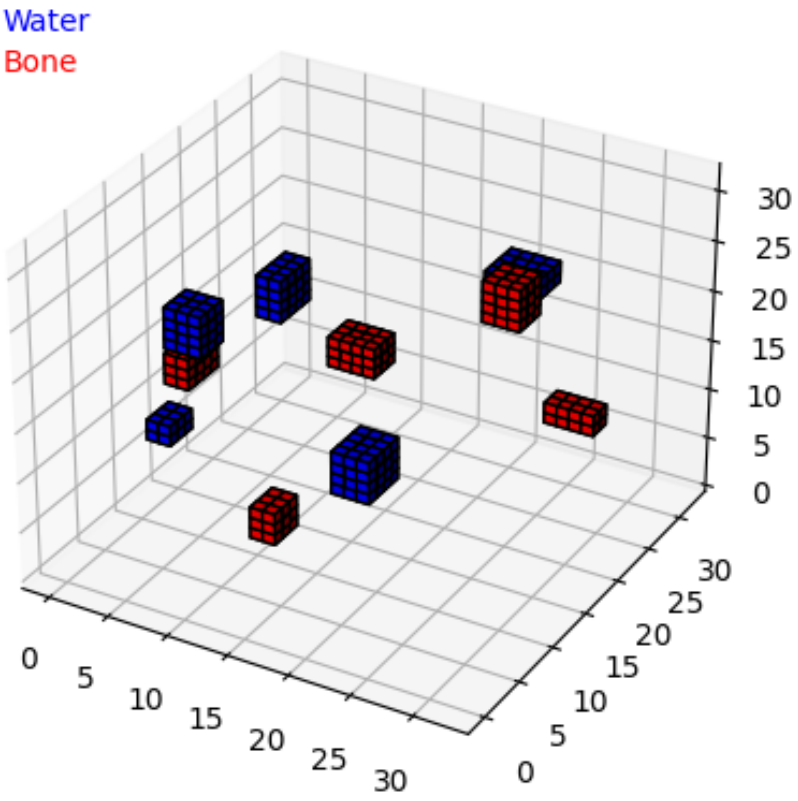
```
import numpy as np
import matplotlib.pyplot as plt
from libs.simulatepreps import projectMatrix
```

3.1 Generate matrices

The matrices will consist of squares of bone and water scattered across the image. This is done to pre-train the network by mapping pixels onto the sinogram.

```
# Generate 32 x 32 x 32 phantom with random squares of water and bone.
# The sizes of the squares can vary between min_size and max_size.
# The value of the bone and water can vary between 0.5 and 1.0.

def generate_phantom(min_size, max_size, number_of_squares):

    water = np.zeros((32, 32, 32))
    bone = np.zeros((32, 32, 32))

    # fill water
    for i in range(number_of_squares):

        # Random size for each square
        water_sizes = np.random.randint(min_size, max_size, size=3)
        water_x, water_y, water_z = np.random.randint(0, 32 - np.array(water_sizes), size=3)

        water[water_x:water_x + water_sizes[0], water_y:water_y + water_sizes[1], water_z:water_z + wat

        bone_sizes = np.random.randint(min_size, max_size, size=3)
        bone_x, bone_y, bone_z = np.random.randint(0, 32 - np.array(bone_sizes), size=3)
        bone[bone_x:bone_x + bone_sizes[0], bone_y:bone_y + bone_sizes[1], bone_z:bone_z + bone_sizes[2

    return  bone, water

# Settings

projections = 32                        # Number of projections
nSubset = 1                             # Number of subsets (based on meeting: change to 1 if algorithm
objectSize = 32                         # Object Size a square
nIter = 20                              # Number of iterations for mechlem algorithm
nPixelsY = 64                           # Number of pixels on the detector plate.
nPixelsZ = 44                           # Number of pixels on the detector plate.
pixelPitch = 1                          # Pixel pitch
nPixelsPerProj = nPixelsZ * nPixelsY    # The total number of pixels on the detector plate

bone, water = generate_phantom(2, 5, 5)
```

```
x = np.column_stack((bone.ravel(), water.ravel()))  # x is the input matrix
y, _, _, _ = projectMatrix(x, objectSize, nPixelsY, nPixelsZ, pixelPitch, projections)

import numpy as np
import matplotlib.pyplot as plt

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

# Create a 3D boolean mask for each material
water_mask = water != 0
bone_mask = bone != 0

# Combine masks into one grid
voxels = water_mask | bone_mask

# Assign colors: use a dict with (z, y, x): color
colors = np.empty(voxels.shape, dtype=object)
colors[water_mask] = 'blue'
colors[bone_mask] = 'red'

ax.voxels(voxels, facecolors=colors, edgecolor='k')

# add legend
ax.text2D(0.05, 0.95, "Water", transform=ax.transAxes, color='blue')
ax.text2D(0.05, 0.90, "Bone", transform=ax.transAxes, color='red')

plt.show()
```


Water
Bone
0
5
10
15
20
25
30
0
5
10
15
20
25
30
0
5
10
15
20
25
30

4 Structuring the data

- append image to sinogram
- fix dimension issues using 0 padding
- fix multiple of 16 issues using 0 padding

As input data we will append the image to the sinogram. There will be a mismatch in height which we will solve by using 0 padding. Due to the convolution layers in the network, it is desired for the input dimensions to be multiples of 16.

```
import numpy as np
from libs.simulatepreps import projectMatrix
```

4.1 Define phantom and generate projection matrix

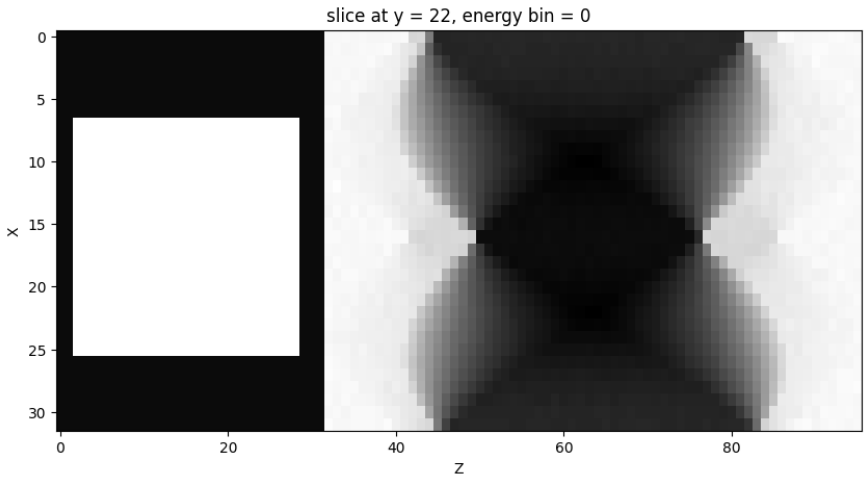
```
# generate a bone and water phantom with a 32x32 size
## SETTINGS ##
objectSize = 32                          # Object Size a square
nPixelsY = 64                            # Number of pixels on the detector plate.
nPixelsZ = 44                            # Number of pixels on the detector plate.
pixelPitch = 1                           # Pixel pitch
nPixelsPerProj = nPixelsZ * nPixelsY     # The total number of pixels on the detector plate
projections = 32                         # Number of projections


def generatePhantom(size):
    bone_cylinder = np.full((size, size, size), 0.05, dtype=np.float32)
    water_cylinder = np.full((size, size, size), 0.05, dtype=np.float32)

    radius = 0.3 * size
    center = size // 2

    for x in range(size):
        for y in range(size):
            for z in range(size):
                if ((x - center) ** 2 + (y - center) ** 2)<= radius ** 2:
                    if (z > 2 and z < size - 2):
                        bone_cylinder[x, y, z] = 1.0
                        water_cylinder[x, y, z] = 1.0

    return bone_cylinder, water_cylinder

bone, water = generatePhantom(objectSize)
x = np.column_stack((bone.ravel(), water.ravel()))
y, _, _, _ = projectMatrix(x, objectSize, nPixelsY, nPixelsZ, pixelPitch, projections)
```

4.2 define prepareInput and prepareOutput

```
def prepareInput(y):

    # Curriculum learning input preparation function
    # As an input: 10 channels consisting of empty images next to the sinograms
```

```
    # shape: 32x48x96, beginning with a 32x32x32 zero image with padding and a 32x44x64 image with padd

    zeroImage = np.zeros((objectSize, objectSize + 16, objectSize))
    zeroBin = np.zeros((projections, 2, nPixelsY))

    inputs = []

    # we have one dataset per y! with 10 channels (one for bone and one for water)
    for i, bins in enumerate(y):
        # no that is not correct

        # input is 10 channels, zeroimage appended to the energy bin
        bin = bins.reshape((projections, nPixelsZ, nPixelsY))

        # normalize so biggest value is 1, smalles value is 0
        bin = (bin - np.min(bin)) / (np.max(bin) - np.min(bin))

        bin = np.concatenate((zeroBin, bin, zeroBin), axis=1)
        # for input image, concatinate bin to zeroImage in x direction
        input_image = np.concatenate((zeroImage, bin), axis=2)
        inputs.append(input_image)
        inputs.append(input_image)

    return inputs

def prepareOutput(water, bone):

    water = np.flip(water, axis=1)  # flip along z -axis
    water = np.flip(water, axis=2)  # flip along y -axis
    bone = np.flip(bone, axis=1)    # flip along z -axis
    bone = np.flip(bone, axis=2)    # flip along y -axis

    zeroImage = np.zeros((32, 8, 32))
    zeroBin = np.zeros((projections, nPixelsZ + 4, nPixelsY))  # zero bin for the energy bin
    outputs = []

    bone_output = np.concatenate((zeroImage, bone, zeroImage), axis=1)
    water_output = np.concatenate((zeroImage, water, zeroImage), axis=1)

    bone_output = np.concatenate((bone_output, zeroBin), axis=2)
    water_output = np.concatenate((water_output, zeroBin), axis=2)

    outputs = [bone_output, water_output]  # shape will be (10, 32, 44, 64)

    return outputs

# Prepare the input and output data
input = prepareInput(y)
output = prepareOutput(water, bone)

from ipywidgets import interact, IntSlider
import matplotlib.pyplot as plt

energy_bin = 0
```

```
#invert the matrix, so z = 0 becomes z = max and y = 0 becomes y = max
flipped_bone_square = np.flip(bone, axis=1)  # flip along z -axis
flipped_bone_square = np.flip(flipped_bone_square, axis=2)  # flip along z -axis


#in input_square[0], replace [:, 8:40, :32] with the phantom
input[energy_bin][:, 8:40, :32] = flipped_bone_square[:, :, :]

# visualize with a slider, put slider for y value

plt.figure(figsize=(10, 5))
plt.imshow(input[energy_bin][:, 22, :], cmap='gray', aspect='auto')
plt.title(f'slice at y = 22, energy bin = {energy_bin}')
plt.xlabel('Z')
plt.ylabel('X')

# draw red rectangle around bottom and top padding
# draw red red rectangle from (0, 0) to (32, 8) and (0, 40) to (32, 48)
plt.show()
```



5 Attention map

An important part of analysing the data is the attention mechanism, which allows the model to focus on different parts of the input data. Being able to visualise this mechanism is a key aspect of understanding how the network behaves. In this notebook I will

- Load preselected data
- Run the model on the preselected data
- Visualise the attention map on the input data

```
from libs.network.network import AttU_Net
import pickle
import os
import torch
import matplotlib.pyplot as plt
import torch.nn.functional as F

# Define paths
network_path = "data/network_att_map.pth"
data_path = "data/data_att_map.pkl"

# load torch input tensor
with open(data_path, "rb") as f:
    input_tensor = pickle.load(f)
```

```
torch.Size([8, 10, 32, 48, 96])
```

5.0.1 Load network and run output

```
network = AttU_Net()
network.load_state_dict(torch.load(network_path, map_location=torch.device('cpu')))  # Load the trained
network.eval()

output, attention_maps = network.getAttenuationMap(input_tensor)
```

5.1 Visualise attention mechanism

Define function show_attention_overlay which as an argument takes the input data, the attention weights, the slice axis and the slice index. For this example I have chosen to slice along the y-direction to properly visualise the sinogram.

```
def show_attention_overlay(input_tensor, attention, slice_axis=2, slice_idx=0, title="Attention Overlay
    """
    Visualize attention over an input slice.

    Args:
        input_tensor: torch.Tensor [1, C, D, H, W]
        attention:    torch.Tensor [1, 1, D', H', W']
        slice_axis:   0=D, 1=H, 2=W (which axis to slice through)
        slice_idx:    index along that axis
    """
    assert input_tensor.ndim == 5 and attention.ndim == 5

    # Resize attention to match input spatial dims
```

```
    attn_resized = F.interpolate(attention, size=input_tensor.shape[2:], mode='trilinear', align_corners

    # Select first input channel and remove batch dim
    input_tensor = input_tensor[6, 0].detach().cpu()
    attn_tensor = attn_resized[6, 0].detach().cpu()

    # Normalize attention
    attn_tensor = (attn_tensor - attn_tensor.min()) / (attn_tensor.max() - attn_tensor.min() + 1e -8)

    # Choose slice
    if slice_axis == 0:  # Depth
        input_slice = input_tensor[slice_idx, :, :].numpy()
        attn_slice  = attn_tensor[slice_idx, :, :].numpy()
    elif slice_axis == 1:  # Height
        input_slice = input_tensor[:, slice_idx, :].numpy()
        attn_slice  = attn_tensor[:, slice_idx, :].numpy()
    elif slice_axis == 2:  # Width
        input_slice = input_tensor[:, :, slice_idx].numpy()
        attn_slice  = attn_tensor[:, :, slice_idx].numpy()
    else:
        raise ValueError("slice_axis must be 0 (D), 1 (H), or 2 (W)")

    plt.figure(figsize=(10, 4))
    # Plot overlay
    plt.imshow(input_slice, cmap='gray')
    plt.imshow(attn_slice, cmap='jet', alpha=0.3)
    # make colourbar same height as image
    plt.xlabel('Z')
    plt.ylabel('X')
    plt.colorbar(label='Attention coefficient', fraction=0.046, pad=0.04)
    plt.title(title + f" (axis={slice_axis}, idx={slice_idx})")
    plt.show()
```

5.1.1 Plot attention map

```
attention = attention_maps['att2']
show_attention_overlay(input_tensor, attention, slice_axis=1, slice_idx=41, title="Attention from att4"
```